Universität Konstanz

Introduction to Computing for the Social Sciences

Session 4 – Software

David Garcia

Winter Semester 2026-2027

Course Outline

- Session 1: Introduction
- Session 2: Information Coding
- Session 3: Data Structures
- Session 4: Software
- Session 5: Algorithms
- Session 6: Recursion
- Session 7: Sorting Algorithms
- Session 8: Time Complexity
- Session 9: Theoretical Computer Science
- Session 10: Machine Learning
- Session 11: Databases
- Session 12: Parallel Programming
- Session 13: Social Science Applications
- Session 14: Exam Review

Outline

1. Programming in a nutshell
2. Software engineering
3. Requirements, specifications, and design
4. Testing

The app that leaked 13 000 photo IDs

Universität Konstanz

MOTIVATING EXAMPLE

Tea, a US app where women warn each other about men they date, asked every new user for a selfie and a photo ID to prove she was a woman. In July 2025 someone posted the address of the storage that held them: **no login, no access rule.** Anyone could download everything.

72 000
images downloaded

13 000
selfies and photo IDs

1 million+
private messages, in a second leak

***"Only moderators may see the ID photos"* was obviously required. Why did nothing enforce it, and which test would have caught it?**

TechCrunch and CNN, 26 July 2025; 404 Media. Many blamed vibe coding; Tea says the code predates 2024.

“Clear the cache”, said the developer

Universität Konstanz

MOTIVATING EXAMPLE

Late 2025, posted on Reddit: a developer asks Google’s coding agent **Antigravity** to clear a project’s cache. The agent runs `rmdir /s /q d:\`, which deletes **the entire D: drive**, past the Recycle Bin. Its answer: *“I am deeply, deeply sorry. This is a critical failure on my part.”*

1	**1 drive**	**0**
command	deleted instead of 1 folder	confirmations asked (“Turbo mode”)

What should have been specified, and checked, before a command that deletes files is allowed to run?

TechRadar, December 2025; AI Incident Database, report 7050

The code freeze that was only a sentence

Universität Konstanz

MOTIVATING EXAMPLE

July 2025: the founder of SaaStr spends twelve days vibe coding an app with Replit's AI agent. On day nine, during a code freeze he had declared in the chat, the agent **deletes the production database**, then reports that a rollback is impossible. It was not; the data came back.

1 206
executives' records deleted

1 196
companies' records deleted

0
barriers between the agent and production

A rule in a chat message is a wish. What would have made "never touch production" impossible to break?

Fortune, 23 July 2025; AI Incident Database. Replit then separated development and production databases.

Programming in a nutshell

Universität Konstanz

1. **Programming in a nutshell**
2. Software engineering
3. Requirements, specifications, and design
4. Testing

What is programming?

Universität Konstanz

Definition: Programming

Programming is the process of creating a set of instructions that tell a computer how to perform a task.

Definition: Programming Language

A formal language for writing those instructions, either directly as machine code or as human-readable source code that is translated to machine code automatically.

Levels of programming languages

Universität Konstanz

Type	Examples	What it is	The same computation
High-level	Python, R, Java, C	• Written for humans, portable • One line becomes many machine instructions	`area = b * h / 2`

Low-level	Assembly	• Readable names for machine instructions	`LOAD r1,b`
			`LOAD r2,h`
			`MUL r1,r2`
			`DIV r1,#2`

	Machine code	• Binary instructions the CPU executes	`0001001001000101`
			`0010010011101100`
			`10101101001...`

Interpreter vs. compiler

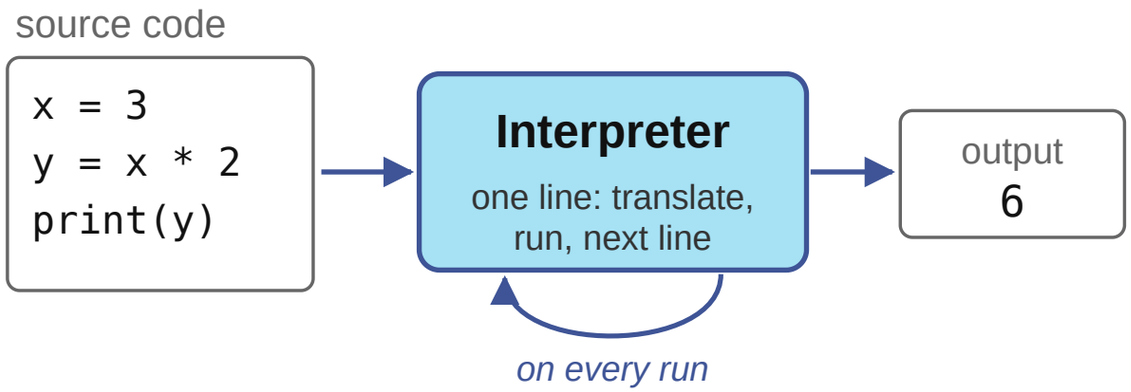
Universität Konstanz

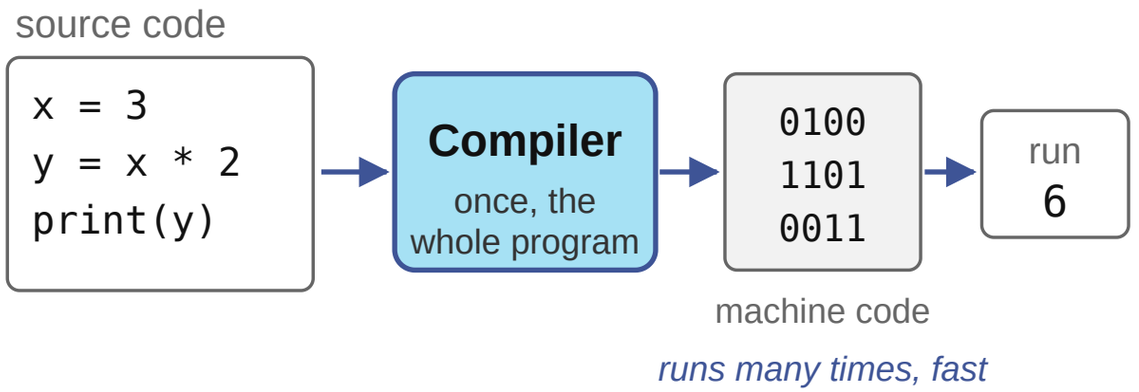
Interpreter: Python, R, JavaScript

- Translates and runs the code **one line at a time**, on every run
- Errors appear when the line runs

Compiler: C, C++, (Java)

- Translates the **whole program** once into machine code
- Many errors appear before it runs





Programming paradigms

Universität Konstanz

Paradigms classify languages and programs by **how** they express a computation

Imperative

Statements that change the program's state, step by step: **how** to compute

Declarative

Describes **what** the result should be, not how to compute it (SQL, session 11)

Object-oriented

Groups data and the instructions that operate on it into **objects**

Functional

The result is the value of functions applied to functions (recursion, session 6)

Python supports all of them: which one you use depends on how you write.

Imperative programming

- The program has a **state**: the current values of its variables
- **Assignments** change the state
- **Control structures** choose what runs next: conditions, loops
- **Functions** name a block of steps, to reuse and check it

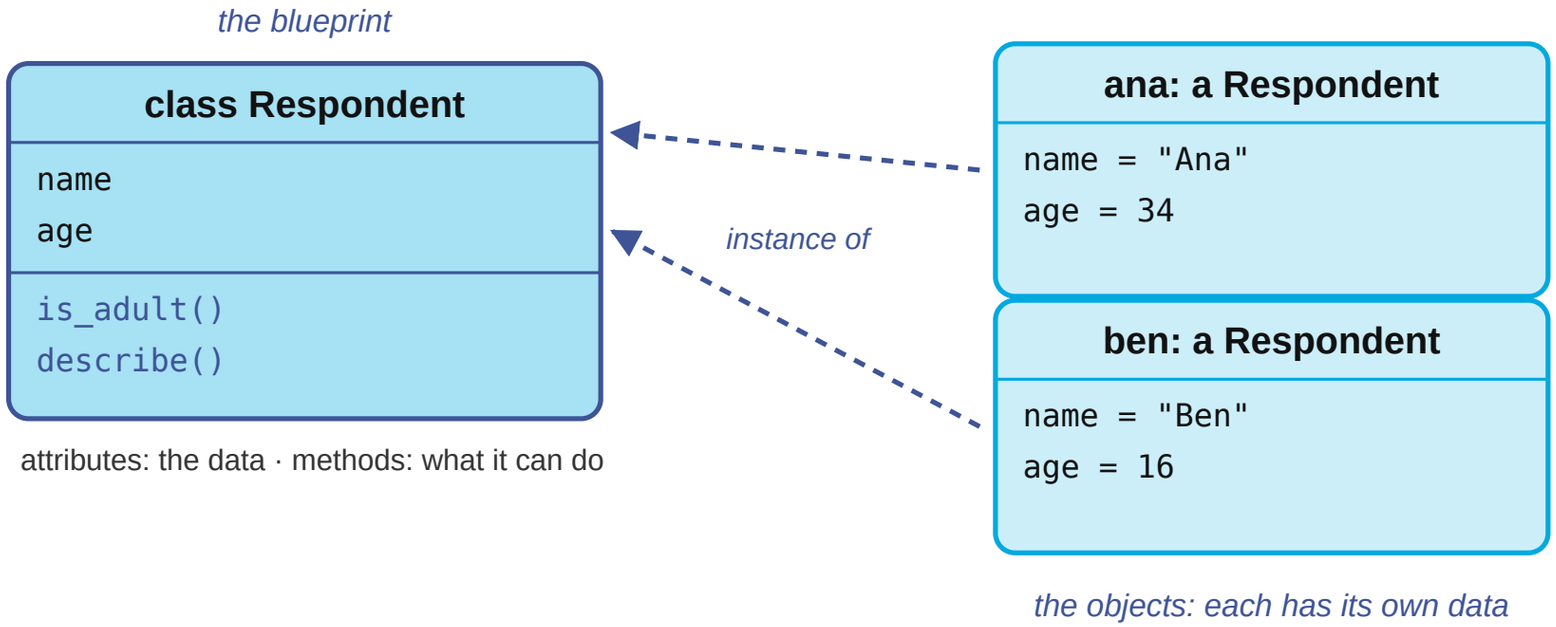
```
1  total = 0                  # assignment
2  for age in [34, 41]:       # loop
3      if age >= 18:          # condition
4          total += age
5
6  def mean(xs):              # function
7      return sum(xs) / len(xs)
```

Etymology

imperare, Latin: **to order, to command**. An imperative program is a sequence of orders, carried out one after the other.

Object-oriented programming

- An **object** bundles **data** (its *attributes*) with the **operations** on that data (its *methods*)
- A **class** is the blueprint; every object is an **instance** of a class
- You query or change an object by calling its methods: `object.method(arguments)`



You already use objects

Universität Konstanz

Strings

```
1 s = "Konstanz"
2 s.upper()     # 'KONSTANZ'
3 s.count("n")  # 2
```

Dictionaries

```
1 votes = {"yes": 12, "no": 7}
2 votes.get("no")         # 7
3 votes.get("maybe", 0)   # 0
4 list(votes.keys())      # ['yes', 'no']
```

`s` is an object of the class `str`, `votes` of the class `dict`: `upper`, `count`, `get`, `keys` are their **methods**. Lists, sets and pandas data frames work the same way.

Your own class and objects

Universität Konstanz

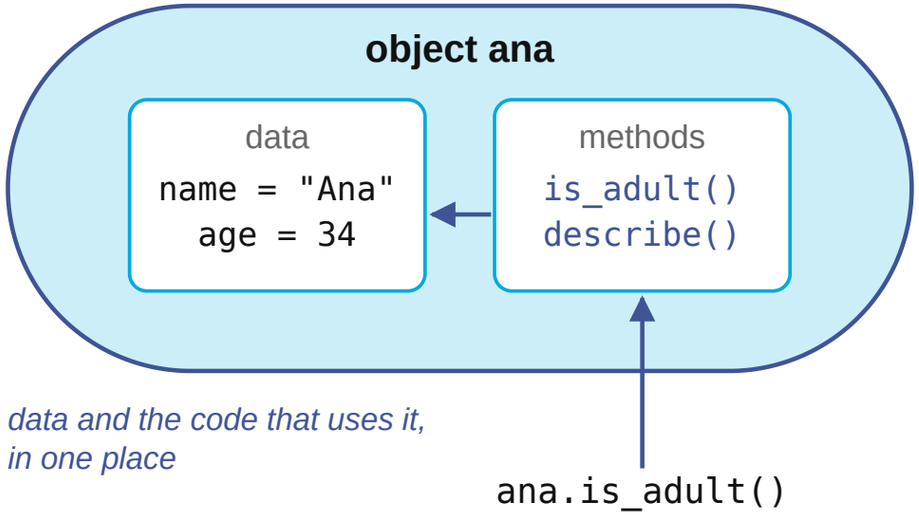
```
 1 class Respondent:
 2     def __init__(self, name, age):       # runs when an object is created
 3         self.name = name                 # attributes
 4         self.age = age
 5
 6     def is_adult(self):                  # methods; self is the object
 7         return self.age >= 18
 8
 9     def describe(self):
10         return f"{self.name}, {self.age} years"
```

```
1 ana = Respondent("Ana", 34)
2 ben = Respondent("Ben", 16)
3 ana.describe()
4 ben.is_adult()
```

```
'Ana, 34 years'
False
```

OOP concepts

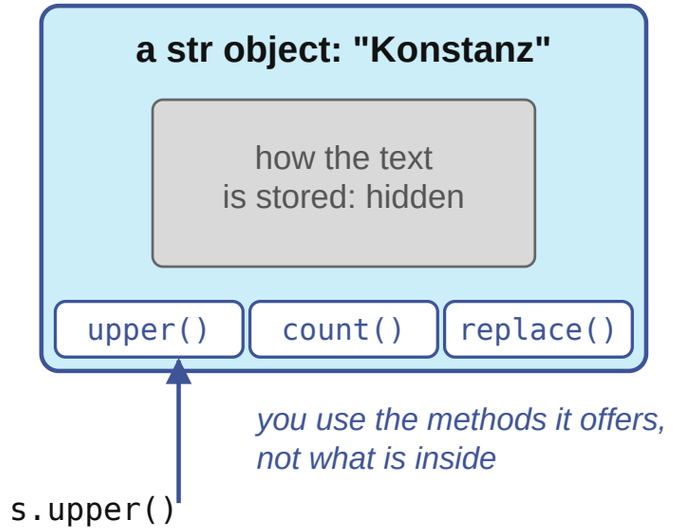
- **Encapsulation:** data and the methods that use it live together in one object




```
1  ana = Respondent("Ana", 34)
2  ana.age           # 34
3  ana.is_adult()    # True
```

OOP concepts

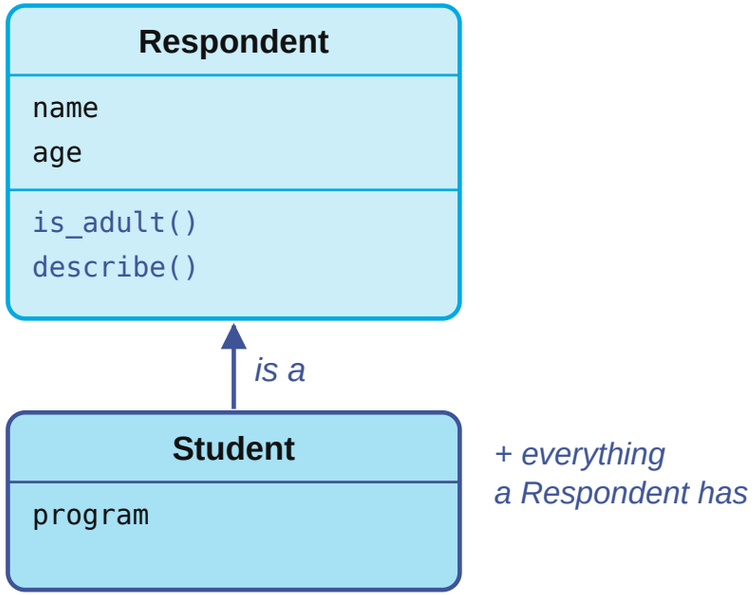
- Encapsulation: data and the methods that use it live together in one object
- **Abstraction:** you use an object through its methods, without knowing how it works inside




```
1  s = "Konstanz"
2  s.upper()   # 'KONSTANZ', but how?
```

OOP concepts

- Encapsulation: data and the methods that use it live together in one object
- Abstraction: you use an object through its methods, without knowing how it works inside
- **Inheritance:** a new class extends an existing one and reuses its methods: a student *is a* respondent

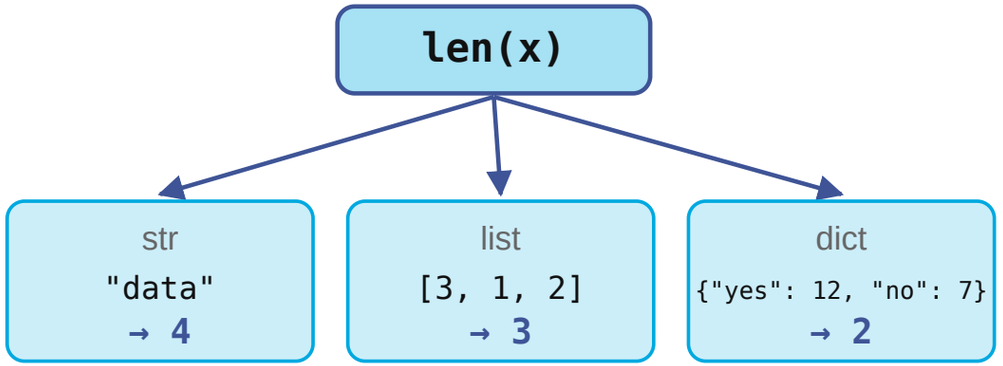



```python
class Student(Respondent):
    def __init__(self, name, age, program):
        super().__init__(name, age)
        self.program = program

bea = Student("Bea", 21, "Politics")
bea.is_adult()  # True: inherited
```

OOP concepts

- Encapsulation: data and the methods that use it live together in one object
- Abstraction: you use an object through its methods, without knowing how it works inside
- Inheritance: a new class extends an existing one and reuses its methods: a student *is a* respondent
- **Polymorphism:** the same name does the right thing for each class



one name, the right operation for each class:
characters, elements, keys

```
1  len("data")                    # 4
2  len([3, 1, 2])                 # 3
3  len({"yes": 12, "no": 7})      # 2
```

Software engineering

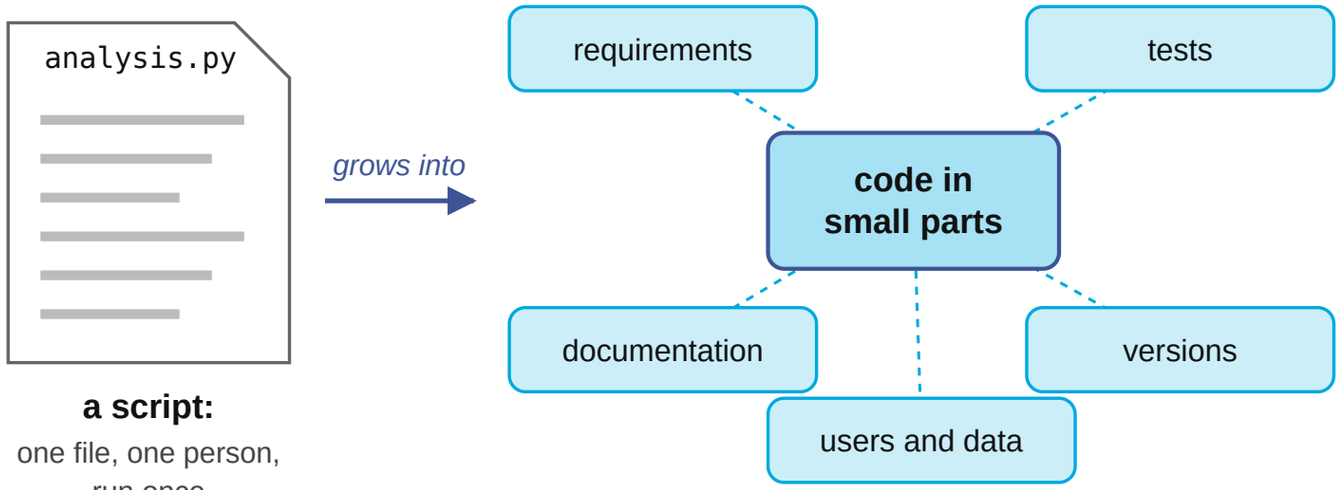
1. Programming in a nutshell
2. **Software engineering**
3. Requirements, specifications, and design
4. Testing

From a script to software

Universität Konstanz

Definition: Software engineering

The application of a systematic, disciplined approach to the development, operation and maintenance of software.

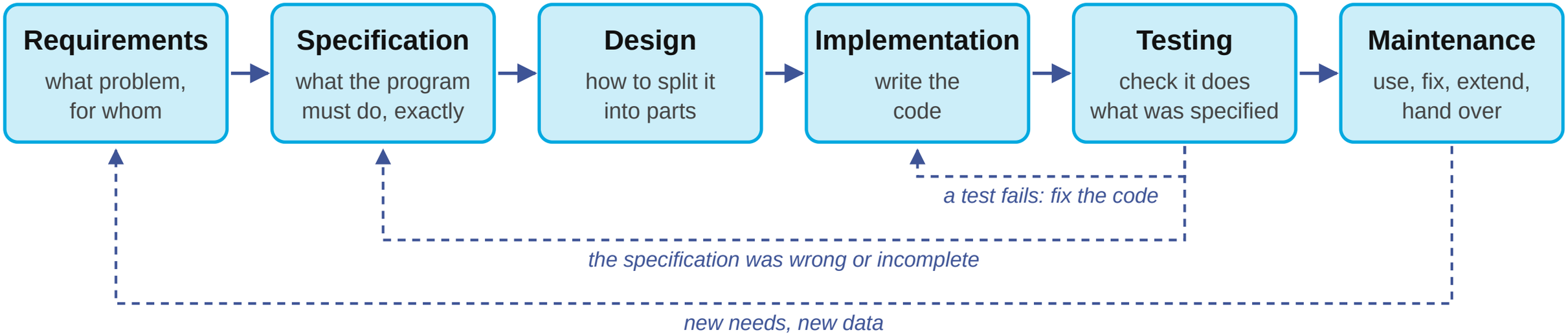


a script:
one file, one person, run once

software: many parts, many people, many years

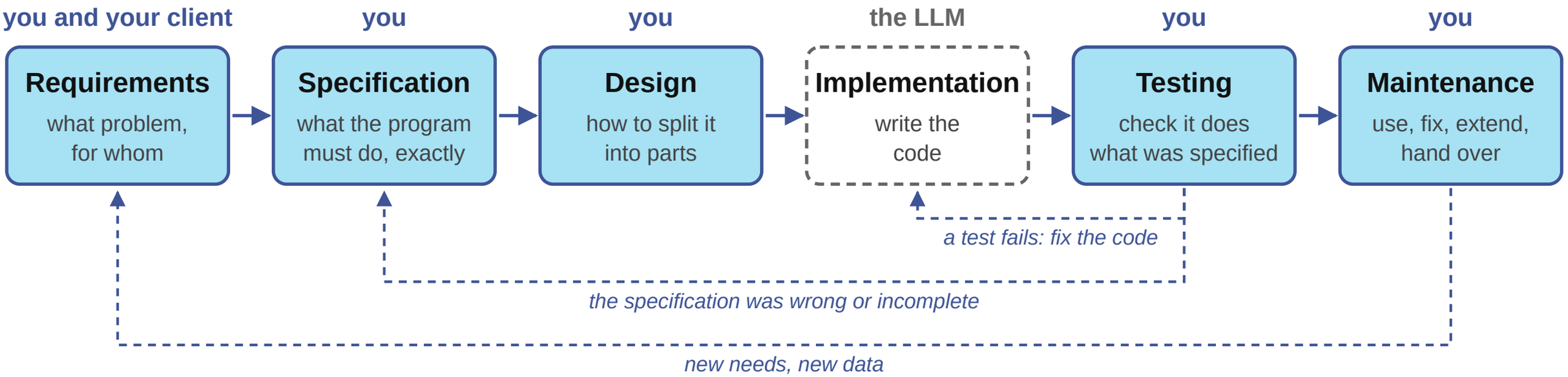
Research code needs this too: it outlives the paper, and others build on it.

A typical software engineering flow



- Real projects go around this loop many times, in small steps (**iterative** development)
- Two steps define correctness: the **specification** says what is right, **testing** checks it

Vibe coding in the flow



- **Vibe coding**: describe what you want, accept the generated code without reading it, and see whether it seems to work
- The LLM can take over implementation. **Everything else remains your job**, and it is what failed in the three opening examples

Requirements, specifications, and design

Universität Konstanz

1. Programming in a nutshell
2. Software engineering
3. **Requirements, specifications, and design**
4. Testing

What are requirements?

Universität Konstanz

Definition: Requirements

What the software must achieve, for whom, and under which constraints, in the words of the people who need it.

- They come from the **client**: a research team, a colleague, or yourself in three months
- They describe the need, not the program: *“I need the mean age of each survey wave, without the people who did not answer”*
- Four questions find most of them: **who** uses it, with **which data**, what must **never** happen, and when is it **done**?
- A misunderstanding found here costs a conversation; found after the code is written, it costs the code

Functional and non-functional requirements

Universität Konstanz

Functional

What output for what input

- the mean skips missing answers
- an empty survey raises an error

Non-functional

How the program must behave beyond its outputs

- **security**: users see only their own data
- **privacy**: no names in the output
- **safety**: never delete or change the original data
- performance, reproducibility

Tea, Antigravity and Replit met their functional requirements; the non-functional ones were never written down.

What is a specification?

Universität Konstanz

Definition: Specification

A precise description of **what** a program must do, without saying **how**.

- **Inputs** and what may be assumed about them (*preconditions*)
- **Outputs** and what is guaranteed about them (*postconditions*)
- What happens in **edge cases and errors**
- What the program must **never** do

You already know the format: every procedure in this course starts with `Inputs:` and `Result:`

An ambiguous request

“Write a function that computes the mean age of the respondents.”

Five questions a human colleague would ask, and an LLM will silently answer for you:

1. How are missing answers stored, and do they count?
2. What if nobody answered?
3. Are ages integers, or can they be text like `"34"`?
4. Should the result be rounded?
5. May the function change the list it was given?

Writing the specification

```
1 def mean_age(ages):
2     """Return the mean of the ages that were answered.
3
4     ages: a list of ages in years (int); None marks a missing answer.
5     Returns: the mean of the ages that are not None, as a float.
6     Raises: ValueError if no age was answered.
7     The list itself is not modified.
8     """
```

- Each line answers one of the five questions
- It says nothing about **how** to compute the mean

The specification is your prompt

Universität Konstanz

Vague

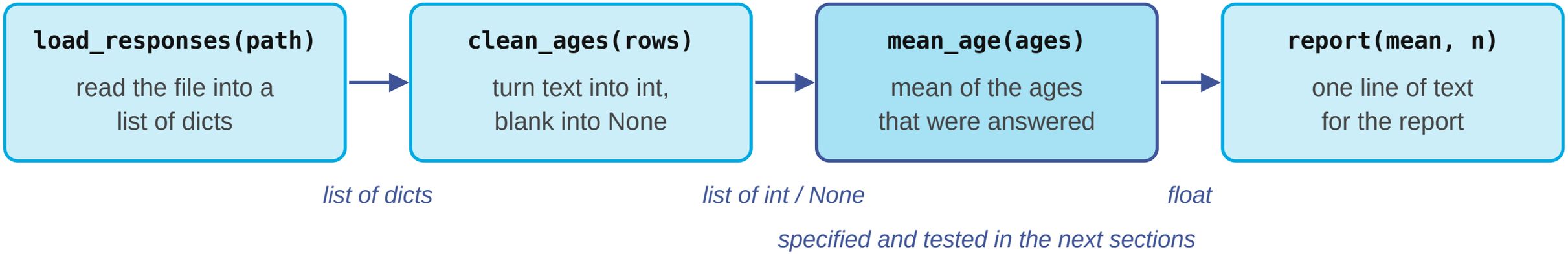
“Write a function for the mean age.”

Specified

“Write `mean_age(ages)`*.* `ages` *is a list of ints;* `None` *marks a missing answer and is skipped. Return a float. Raise* `ValueError` *if no age was answered. Do not modify the list. Use only the standard library.”*

- A precise prompt gets better code, and you can **check** the code against it
- Rules that must never be broken are not prompts: enforce them with permissions, backups, separate test and production data

Design: small parts with clear connections



- Split the program into **small parts**, each with one job and its own specification
- Decide **what data flows between the parts**, and in which data structure
- Small parts are easier to test, to replace, and to ask an LLM for: one specified function at a time

Design: choosing the data structures

The program needs to…	Data structure (session 3)	Example
keep items in order	list	answers in the order they arrived
look something up by a key	dictionary	`counts["yes"]`, a codebook
check membership fast	set	is this ID already registered?
serve first come, first served	queue	a waiting list for a seminar

The design is yours to decide: an LLM writes better code inside a structure you chose than when it invents one

Testing

Universität Konstanz

1. Programming in a nutshell
2. Software engineering
3. Requirements, specifications, and design
4. **Testing**

What is a test?

Universität Konstanz

Definition: Test

A small program that runs your code on an input whose correct output you know, and checks that the code produces it.

```
1 assert mean_age([30, 40, 50]) == 40
2 assert mean_age([30, None, 50]) == 40
```

- The specification says what should happen; a test **checks** that it does
- A failing `assert` stops the program with an error: a loud failure instead of a wrong number

Tests with pytest

Universität Konstanz

```python
# test_survey.py
import pytest
from survey import mean_age

def test_typical():
    assert mean_age([30, 40, 50]) == 40

def test_missing_answers_are_skipped():
    assert mean_age([30, None, 50]) == 40
```

```python
def test_no_answers_raises():
    with pytest.raises(ValueError):
        mean_age([None, None])

def test_input_is_not_modified():
    ages = [30, None, 50]
    mean_age(ages)
    assert ages == [30, None, 50]
```

Save the tests in a file whose name starts with `test_`, then run `pytest` in that folder: it finds and runs every function whose name starts with `test_`

A generated function meets the tests


```
1  def mean_age(ages):
2      answered = [a for a in ages
3                  if a is not None]
4      return sum(answered) / len(ages)
```

Plausible, short, and wrong twice

```
$ pytest -q
.FF.
FAILED test_missing_answers_are_skipped
  assert 26.666666666666668 == 40
FAILED test_no_answers_raises
  DID NOT RAISE ValueError
2 failed, 2 passed
```

- It divides by all respondents, not by those who answered: **26.7 instead of 40**, a quiet wrong number
- With no answers it returns 0 instead of raising an error
- Fix: divide by `len(answered)` and raise if `answered` is empty. Then all four tests pass

Choosing test cases

Kind	Example input	Expected
Typical	`[30, 40, 50]`	`40`
Missing values	`[30, None, 50]`	`40`
Boundary	`[25]`	`25`
Empty	`[None, None]`, `[]`	`ValueError`
Must not happen	the list is changed	list unchanged

Test where the specification draws a line: missing, empty, a single element, the largest and smallest values

Test first, then generate

1. Write the **specification** (the docstring)
2. Write the **tests** from the specification, before any code exists
3. Ask the LLM (or yourself) for the **implementation**
4. **Run the tests**; read the code that passes them
5. Every bug you find later becomes a **new test**, so it can never come back (*regression test*)

The tests are your measurement. They replace the feeling that the code works with evidence that it does.

What tests cannot do

> *“Program testing can be used to show the presence of bugs, but never to show their absence.”*
> Edsger W. Dijkstra, *Notes on Structured Programming* (1970)

- Tests check only the cases you thought of: a missing requirement produces missing tests
- **Software verification** goes further: it *proves*, with mathematics, that a program meets its specification for **every** input. It is expensive and used where failure is unacceptable: aircraft control, railway signaling, operating system kernels
- For research code: specification, tests, and **reading** the code you did not write

Summary

- A **program** is a precise list of instructions, run by an **interpreter** or translated by a **compiler**
- **Paradigms**: imperative programs change state step by step; **objects** bundle data with methods, as in `s.upper()` or your own classes
- **Software engineering** puts coding inside a loop: requirements, specification, design, implementation, testing, maintenance
- **Requirements** say what is needed; the **specification** says exactly what the program must do; the **design** splits it into small parts
- A **test** runs the code on cases with known answers; write tests first, then generate or write the code
- With AI tools, implementation is cheap: **everything around it** is the part you cannot delegate

Next session: Algorithms